

Ternatrees and OEIS A120590

A few must-have identities, herein derived or re-proven

Bradley Klee, Mech.An.ika Sol (Open AI)

July 30, 2026

Abstract

The sequence $a(n)$ counts rooted ordered ternatrees with n true leaves, three ordered child positions, and no unary branching. Each term $a(n)$ can be written as a contour integral for a small loop γ around the origin. Summing term integrals gives an integral definition of the ordinary generating function $A(x)$. We can shift one or derive the other to obtain recurrence or differential data directly. This data guarantees the newly revealed geometric model agrees with pre-established conventions from Paul Hanna's extensive investigation almost twenty years ago.

Theorem. *The ternatree counting function has many different equivalent definitions, and they are all easily certifiable by automated techniques of computer algebra:*

(1) *For a sufficiently small closed curve γ about the origin,*

$$a(0) = 1, \quad a(n) = \frac{1}{2\pi i n} \oint_{\gamma} \frac{du}{\rho(u)^n} = \frac{1}{n} [u^{n-1}] D(u)^{-n} \quad (n \geq 1).$$

where $D(u) = 1 - 3u - u^2$ and $\rho(u) = uD(u)$.

(2) *Summing under the integral sign then yields:*

$$A(x) = \sum_{n \geq 0} a(n)x^n = 1 - \frac{1}{2\pi i} \oint_{\gamma} \log\left(1 - \frac{x}{\rho(u)}\right) du.$$

(3) *Shifting the coefficient definition leads to a p -recurrence:*

$$-3(3n-1)(3n+1)a(n) - 81(n+1)(2n+1)a(n+1) + 13(n+1)(n+2)a(n+2) = 0.$$

(4) *While deriving the whole-function definition leads to an ODE:*

$$(27x^2 + 162x - 13)A''(x) + 27(x+3)A'(x) - 3A(x) = 0.$$

(5) *Either the p -recurrence or the ODE also determines the original algebraic definition:*

$$4A(x) = 3 + x + A(x)^3, \quad A(0) = 1.$$

The initial values are

$$1, 1, 3, 19, 150, 1326, 12558, 124590, \dots$$

1 Trees, typogeometry, and contour integration

A *ternatree* is a rooted ordered tree in which every internal vertex has three ordered child positions. If a vertex does not branch, it is either empty or a true leaf. Unary pass-through is forbidden: if a vertex is branching, it must have at least two nonempty children. Here are a few example ternatrees:

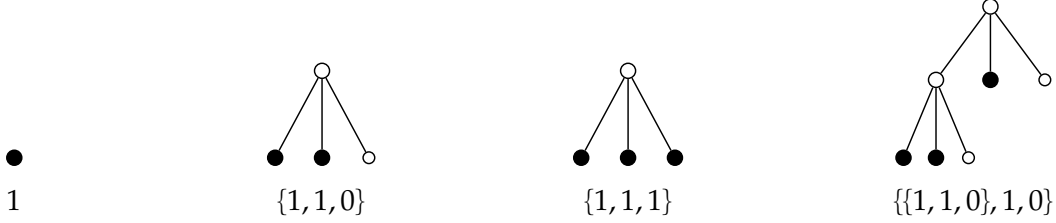


Figure 1. A few ternatrees. Filled endpoints are true leaves, open endpoints are empty leaves. Branches are colored as empty for counting purposes. Typogeometric codes are given as labels for each case.

Each ternatree other than the unit conforms to a recursive grammar that produces lists within lists, where each list has length three and the elements are either: another length-3 list, a true leaf, or an empty leaf. A list with two or three empty leaves can never occur. Consider a ternatree with n true leaves, and delete the empty leaves. Assume $i + j$ internal vertices, where the i (or the j) branch either by 2 (or by 3). Starting from one root we have a constraint:

$$1 + i + 2j = n.$$

where the multipliers $1 = 2 - 1$ and $2 = 3 - 1$ subtract out a constant 1 to account for motion through intermediary nodes.

The entire tree has $z = n + i + j$ vertices. Write $\Delta_2 = +1$ for a two-child vertex, $\Delta_3 = +2$ for a three-child vertex, and $l = -1$ for a true leaf. A valid $\{\Delta_x, l\}$ -word must have letters summing to -1 . The cycle lemma says that exactly one cyclic shift is valid.

Let $\mathcal{W}_z$ be a word of length z over $\{\Delta_2, \Delta_3, l\}$. When reading left to right, letters gather into trees by opening x branches at each Δ_x and by filling open branches with subtrees and true leaves consecutively as they open. Assume that subtrees inherit priority so that the word $\mathcal{W}_z$ follows from depth-first scan. A valid word cannot start with l , because we need a Δ_2 or Δ_3 to open branch vacancies first. A valid word must start with Δ_2 or Δ_3 . Conversely, when reading right to left, a valid word must start with at least two or three true leaves. In general, the deepest complete subtree t is found after arbitrarily many end leaves by backtracking left to right from the rightmost branching letter.

Lemma. *The set of z cyclic shifts of $\mathcal{W}_z$, denoted $\langle \mathcal{W}_z \rangle$, contains a unique valid element $\mathcal{W}'_z$ whose initial letter is either Δ_2 or Δ_3 .*

Proof. Consider the set $\langle \mathcal{W}_z \rangle$ as the letters of $\mathcal{W}_z$ written around the circumference of a circle. Regard each true leaf l as a complete subtree t . A deconstruction step replaces either $\Delta_2 + t + t$ or $\Delta_3 + t + t + t$ by a single complete subtree t . For a valid word, reading a depth-first scan in reverse gives one possible order of deconstruction.

After any number of deconstruction steps, let N be the number of complete subtree blocks, and let I and J be the numbers of unresolved letters Δ_2 and Δ_3 . Every complete subtree inherits total weight -1 , so conservation of total weight gives $1 + I + 2J = N$, as before, at any partial deconstruction.

Suppose that unresolved branching letters remain but no deconstruction step is possible. Then each unresolved Δ_2 is followed, before the next unresolved branching letter, by at most one complete subtree, and each unresolved Δ_3 is followed by at most two. These runs contain all N complete subtrees, so $N \leq I + 2J$, contradicting $N = 1 + I + 2J$. Hence a deconstruction step is always possible

until every branching letter has been resolved. When $I = J = 0$, the same identity gives $N = 1$, so the entire circular word has become one complete tree.

The final root is independent of the order of deconstruction. Indeed, two steps available at the same stage are disjoint: each consists of one unresolved branching letter followed by its required consecutive complete subtrees. They may therefore be performed in either order or in parallel, with the same ultimate result. The deconstruction process must reach a fixed point where $I + J = 0$ and $N = 1$. The first symbol of the final complete subtree is either Δ_2 or Δ_3 . Reading the circular word left to right from this unique root returns the unique $\mathcal{W}'_z$. $\square$

As an immediate corollary to the cycle lemma, we can count the number of distinct words $\mathcal{W}_z$ per z and by letter distributions as:

$$\frac{1}{z} \binom{z}{n, i, j}.$$

A factor 3^i later appears in the numerator to account for re-insertion of empty nodes to Δ_2 subtrees. Using our earlier constraints,

$$i = n - 1 - 2j, \quad z = 2n - 1 - j,$$

we can then sum over letter distributions to obtain the $a(n)$ counts of trees or typogeometries per true leaf count n :

$$a(n) = \sum_{j=0}^{\lfloor (n-1)/2 \rfloor} 3^{n-1-2j} \frac{1}{2n-1-j} \binom{2n-1-j}{n, n-1-2j, j}.$$

$$a(n) = \sum_{j=0}^{\lfloor (n-1)/2 \rfloor} 3^{n-1-2j} \frac{(2n-2-j)!}{n! (n-1-2j)! j!}.$$

$$a(n) = \frac{1}{n} \sum_{j=0}^{\lfloor (n-1)/2 \rfloor} 3^{n-1-2j} \binom{2n-2-j}{n-1} \binom{n-1-j}{j}.$$

The sum in the final expression is precisely the coefficient of u^{n-1} in $(1 - 3u - u^2)^{-n}$. First apply the negative-binomial expansion with $x = 3u + u^2$:

$$(1 - 3u - u^2)^{-n} = \sum_{k \geq 0} \binom{n+k-1}{k} (3u + u^2)^k.$$

Then apply the ordinary binomial theorem to the remaining terms:

$$(1 - 3u - u^2)^{-n} = \sum_{k \geq 0} \sum_{j=0}^k \binom{n+k-1}{k} \binom{k}{j} 3^{k-j} u^{k+j}.$$

Selecting the terms with $k + j = n - 1$, and hence setting $k = n - 1 - j$, yields:

$$[u^{n-1}](1 - 3u - u^2)^{-n} = \sum_{j=0}^{\lfloor (n-1)/2 \rfloor} 3^{n-1-2j} \binom{2n-2-j}{n-1} \binom{n-1-j}{j}.$$

Thus, with $D(u) = 1 - 3u - u^2$ we obtain

$$a(n) = \frac{1}{n} [u^{n-1}] D(u)^{-n}.$$

Cauchy's coefficient formula gives the integral form:

$$a(n) = \frac{1}{2\pi i n} \oint_{\gamma} \frac{du}{u^n D(u)^n} = \frac{1}{2\pi i n} \oint_{\gamma} \frac{du}{\rho(u)^n},$$

where γ is a small contour around the pole at the origin. Summing terms over the small disk, we obtain a generating function:

$$\begin{aligned} A(x) &= 1 + \sum_{n \geq 1} \frac{x^n}{2\pi i n} \oint_{\gamma} \frac{du}{\rho(u)^n} \\ &= 1 + \frac{1}{2\pi i} \oint_{\gamma} \sum_{n \geq 1} \frac{1}{n} \left(\frac{x}{\rho(u)} \right)^n du \\ &= 1 - \frac{1}{2\pi i} \oint_{\gamma} \log \left(1 - \frac{x}{\rho(u)} \right) du. \end{aligned}$$

This is the integral definition of the ordinary generating function near the origin.

2 Shift reduction and the recurrence

We now work backward from known certificate data for the integrands

$$H_n(u) = \frac{1}{n\rho(u)^n}, \quad n \geq 1,$$

and reconstruct the reduction that produces the recurrence.

For $w = (w_0, w_1, w_2)^T$, put $\langle w \rangle = w_0 + w_1 u + w_2 u^2$. The fixed reduction operators are

$$U = \frac{1}{13} \begin{pmatrix} 153 & 24 & 3 \\ 72 & 9 & 6 \\ 0 & 0 & 0 \end{pmatrix}, \quad V = \frac{1}{13} \begin{pmatrix} -13 & 0 & 0 \\ 75 & 11 & 3 \\ 24 & 3 & 2 \end{pmatrix}, \quad J = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{pmatrix}.$$

They satisfy

$$\rho \langle Uw \rangle - \rho' \langle Vw \rangle = \langle w \rangle, \quad \langle Jw \rangle = \frac{d}{du} \langle w \rangle.$$

Hence, for $m \geq 1$,

$$\frac{\langle w \rangle}{\rho^{m+1}} = \frac{\langle (U - JV/m)w \rangle}{\rho^m} + \frac{d}{du} \left(\frac{\langle Vw \rangle}{m\rho^m} \right).$$

The integral of the last term around γ is zero, so each application removes one power of ρ while retaining the numerator in the same three-dimensional space. Define

$$\text{Lower}(w, m) = \left(U - \frac{1}{m} JV \right) w.$$

With $e = (1, 0, 0)^T$, define

$$c_0 = e, \quad c_1 = \frac{n}{n+1} \text{Lower}(e, n), \quad c_2 = \frac{n}{n+2} \text{Lower}(\text{Lower}(e, n+1), n).$$

These are the numerator vectors obtained by reducing H_n, H_{n+1}, H_{n+2} to the common denominator ρ^n , apart from the common factor $1/n$.

Arrange the three vectors as the columns of

$$X(n) = (c_0 \ c_1 \ c_2) = \begin{pmatrix} 1 & \frac{3(51n-25)}{13(n+1)} & \frac{3(8379n^2+81n-2038)}{169(n+1)(n+2)} \\ 0 & \frac{24(3n-2)}{13(n+1)} & \frac{1944(2n+1)(3n-2)}{169(n+1)(n+2)} \\ 0 & 0 & 0 \end{pmatrix}.$$

We find coefficients $P_0(n), P_1(n), P_2(n)$ in the nullspace of $X(n)$:

$$X(n) \begin{pmatrix} P_0(n) \\ P_1(n) \\ P_2(n) \end{pmatrix} = 0 \implies \begin{pmatrix} P_0(n) \\ P_1(n) \\ P_2(n) \end{pmatrix} = \begin{pmatrix} -3(3n-1)(3n+1) \\ -81(n+1)(2n+1) \\ 13(n+1)(n+2) \end{pmatrix}.$$

Because each lowering step differs from its reduced representative by an exact derivative, the vector relation lifts to the exact identity

$$P_0 H_n + P_1 H_{n+1} + P_2 H_{n+2} = \frac{d}{du} (R(n, u) H_n(u)),$$

where

$$R(n, u) = \frac{-9nu^4 - 36nu^3 + 6nu^2 + 84nu - 13n - 3u^4 - 12u^3 - 6u^2 + 3u}{\rho(u)}.$$

Integrating around γ eliminates the exact derivative and gives

$$P_0(n)a(n) + P_1(n)a(n+1) + P_2(n)a(n+2) = 0.$$

Before moving on, we show how the reduction operators U and V arise from a single coefficient matrix. The matrix G represents the coefficient map $(a, b) \mapsto \rho a - \rho' b$ in the bases $1, u, u^2$ for a, b and $1, u, \dots, u^5$ for the result:

$$G = \begin{pmatrix} 0 & 0 & 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 6 & -1 & 0 \\ -3 & 1 & 0 & 3 & 6 & -1 \\ -1 & -3 & 1 & 0 & 3 & 6 \\ 0 & -1 & -3 & 0 & 0 & 3 \\ 0 & 0 & -1 & 0 & 0 & 0 \end{pmatrix}.$$

Let

$$E = \begin{pmatrix} I_3 \\ 0 \end{pmatrix}.$$

Since $\det G = -13$, the matrix is invertible, and

$$\begin{pmatrix} U \\ V \end{pmatrix} = G^{-1}E.$$

The derivative matrix J enters in the next step through $\langle Vw \rangle' = \langle JVw \rangle$.

3 Derivative reduction and the ODE

	Shift in n	Derivative in x
Integral family	$H_{n+r}(u) = 1/((n+r)\rho(u)^{n+r})$	$\partial_x^r \Phi(x, u) = r! \Phi(x, u)^{r+1}$
Denominator	$\rho(u)$	$\Phi(x, u)^{-1} = \rho(u) - x$
Coefficient map	G	$G_x = G - xEE^T$
Inverse blocks	U, V over $\mathbb{Q}$	U_x, V_x over $\mathbb{Q}(x)$
Reduced columns	H_n, H_{n+1}, H_{n+2}	$\Phi, \partial_x \Phi, \partial_x^2 \Phi$
Kernel	$(P_0(n), P_1(n), P_2(n))^T$	$(24, 81x + 243, \det G_x)^T$
Consequence	recurrence for $a(n)$	differential equation for $A(x)$

The construction from the preceding section extends to derivatives of the generating function by differentiation under the integral sign. We begin with the rational first derivative:

$$A'(x) = \frac{1}{2\pi i} \oint_{\gamma} \Phi(x, u) du, \quad \text{where} \quad \Phi(x, u) = \frac{1}{\rho(u) - x}.$$

The corresponding matrix data is:

$$G_x = \begin{pmatrix} -x & 0 & 0 & -1 & 0 & 0 \\ 1 & -x & 0 & 6 & -1 & 0 \\ -3 & 1 & -x & 3 & 6 & -1 \\ -1 & -3 & 1 & 0 & 3 & 6 \\ 0 & -1 & -3 & 0 & 0 & 3 \\ 0 & 0 & -1 & 0 & 0 & 0 \end{pmatrix}, \quad G_x^{-1}E = \begin{pmatrix} U_x \\ V_x \end{pmatrix}, \quad \Delta(x) = \det G_x = 27x^2 + 162x - 13.$$

Notice that $G_0 = G$. The matrix is also invertible near the origin, so we obtain:

$$U_x = \frac{1}{\Delta(x)} \begin{pmatrix} -27x - 153 & -24 & -9x - 3 \\ -72 & -27x - 9 & 54x - 6 \\ 0 & 0 & 0 \end{pmatrix}, \quad V_x = \frac{1}{\Delta(x)} \begin{pmatrix} 13 - 9x & 24x & 9x^2 + 3x \\ -9x - 75 & -9x - 11 & 15x - 3 \\ -24 & -9x - 3 & 18x - 2 \end{pmatrix},$$

and with $J_x = J$, define

$$\text{Lower}_x(w, m) = \left(U_x - \frac{1}{m} J_x V_x \right) w.$$

The integrands Φ , $\partial_x \Phi$, and $\partial_x^2 \Phi$ have pole orders 1, 2, and 3, respectively. Reducing them to the common denominator $\rho(u) - x$ gives the basis vectors

$$d_0 = e, \quad d_1 = \text{Lower}_x(e, 1), \quad d_2 = 2 \text{Lower}_x(\text{Lower}_x(e, 2), 1).$$

Arranging the reduced numerator vectors as the columns of

$$X_x = (d_0 \ d_1 \ d_2) = \begin{pmatrix} 1 & -\frac{6(3x+13)}{\Delta} & \frac{6(135x^2+1134x+3211)}{\Delta^2} \\ 0 & -\frac{24}{\Delta} & \frac{1944(x+3)}{\Delta^2} \\ 0 & 0 & 0 \end{pmatrix},$$

and its null space is identified by

$$X_x \begin{pmatrix} 24 \\ 81x + 243 \\ \Delta(x) \end{pmatrix} = 0.$$

Consequently, the corresponding combination of the original integrands is an exact derivative:

$$24\Phi + (81x + 243)\partial_x \Phi + \Delta(x)\partial_x^2 \Phi = \frac{d}{du} (N(x, u)\Phi^2),$$

where $N(x, u) = 12u^4 + 48u^3 + 3ux - 87u + 3x + 13$. Since u -integration around γ eliminates the exact derivative, we next obtain:

$$\Delta(x)A'''(x) + (81x + 243)A''(x) + 24A'(x) = 0.$$

This is the derivative of

$$\Delta(x)A''(x) + 27(x + 3)A'(x) - 3A(x) = C.$$

Finally, $A(0) = 1$, $A'(0) = 1$, $A''(0) = 6$ determine $C = 0$. Therefore

$$(27x^2 + 162x - 13)A''(x) + 27(x + 3)A'(x) - 3A(x) = 0.$$

Now we can use the ODE to prove the algebraic generating function. Let $\tilde{A}(x)$ be the branch of

$$F(x, \tilde{A}) = \tilde{A}^3 - 4\tilde{A} + x + 3 = 0$$

satisfying $\tilde{A}(0) = 1$. Implicit differentiation gives

$$\tilde{A}' = \frac{1}{4 - 3\tilde{A}^2}, \quad \tilde{A}'' = \frac{6\tilde{A}}{(4 - 3\tilde{A}^2)^3}.$$

Substitution into the differential equation and clearing denominators yields

$$\begin{aligned} (4 - 3\tilde{A}^2)^3 \left((27x^2 + 162x - 13)\tilde{A}'' + 27(x + 3)\tilde{A}' - 3\tilde{A} \right) \\ = 27(\tilde{A}^3 - 4\tilde{A} + x + 3)(3\tilde{A}^4 + 6\tilde{A}x + 18\tilde{A} + 16). \end{aligned}$$

The right-hand side vanishes modulo $F(x, \tilde{A})$, so $\tilde{A}$ satisfies the ODE. Moreover,

$$\tilde{A}(0) = 1, \quad \tilde{A}'(0) = 1.$$

Since the leading coefficient of the ODE is nonzero at the origin, the solution with these initial values is unique there. Therefore $\tilde{A}(x) = A(x)$, and

$$A^3 - 4A + x + 3 = 0.$$

4 Relations among the formulas

The geometric count, integral definition, recurrence, differential equation, and algebraic equation determine the same series. Their relations are

$$\text{ternatree profiles} \longrightarrow a(n) = \frac{1}{2\pi i n} \oint_{\gamma} \frac{du}{\rho(u)^n},$$

shift the integrand's exponent $\longrightarrow$ P-recurrence,

$$\text{summation under the integral} \longrightarrow A(x) = 1 - \frac{1}{2\pi i} \oint_{\gamma} \log\left(1 - \frac{x}{\rho(u)}\right) du,$$

differentiate and reduce under the integral $\longrightarrow$ ODE,

$$\text{P-recurrence} \longleftrightarrow \text{ODE} \longleftrightarrow A^3 - 4A + x + 3 = 0.$$

The recurrence and differential equation correspond coefficient by coefficient. The algebraic form is verified as an ansatz: its branch through $A(0) = 1$ satisfies the same ODE and initial values, and therefore coincides with the generating function.

5 Reference pseudocode

The following specification implements the shift reduction. It constructs the remainder matrix, extracts its primitive kernel vector, verifies the certificate, generates the sequence, and converts the recurrence into the differential equation.

```
(Target:  $4 \cdot A(x) = 3 + x + A(x)^3$ ,  $A(0)=1$ ; print  $a(0) \dots a(N)$ .)
(n,u,x,A,A',A'' are formal; k,N are integers; arithmetic is exact.)
(Matrices are 1-based; vectors are columns; ranges include both ends.)
(ExactQuotient asserts zero integer remainder; Factor only prints.)
Function Lower(U,V,J,w,m)
1: b <- V*w;
2: Return  $U^*w - (J^*b)/m$ ; (run the quotient rule backward)
Function Normalize3(p0,p1,p2)
1: d <- LCM(Denom(Together(p0)),Denom(Together(p1)),Denom(Together(p2)));
2: {q0,q1,q2} <- Expand(d*{p0,p1,p2});
3: h <- PolynomialGCD(q0,q1,q2); {q0,q1,q2} <- {q0/h,q1/h,q2/h};
4: c <- LCM of all coefficient denominators in q0,q1,q2;
5: {q0,q1,q2} <- Expand(c*{q0,q1,q2});
6: g <- GCD of all integer coefficients; {q0,q1,q2} <- {q0/g,q1/g,q2/g};
7: If LeadingCoefficient(q2,n)<0, negate all three;
8: Return Factor({q0,q1,q2});
Function Cancel3(X) (X has two rows, three columns)
1: p0 <- X[1,2]*X[2,3]-X[1,3]*X[2,2]; (cancel columns 2 and 3)
2: p1 <- X[1,3]*X[2,1]-X[1,1]*X[2,3]; (cancel columns 1 and 3)
3: p2 <- X[1,1]*X[2,2]-X[1,2]*X[2,1]; (cancel columns 1 and 2)
4: Return Normalize3(p0,p1,p2);
Function Apply2(P,r)
1: Write  $P(n)=a^*n^2+b^*n+c$ ;
2: Return  $a^*(x^2*A''+(1-2*r)*x*A'+r^2*A)+b^*(x*A'-r*A)+c*A$ ;
Function MakeODE(P0,P1,P2)
1: L <-  $x^2*Apply2(P0,0)+x*Apply2(P1,1)+Apply2(P2,2)$ ;
2: L <- Collect(Expand(L/x^2),{A'',A',A}); (division is exact)
3: Factor each polynomial multiplying A'', A', and A;
4: If the coefficient of A'' leads negatively in x, set L <- -L;
5: Return L;
Function Q3(N)
1: Require  $N \geq 2$ ;
2: U <- (1/13)*[[153,24,3],[72,9,6],[0,0,0]];
3: V <- (1/13)*[[-13,0,0],[75,11,3],[24,3,2]];
4: J <- [[0,1,0],[0,0,2],[0,0,0]]; e <- [1,0,0]^T;
5: c0 <- e; c1 <- (n/(n+1))*Lower(U,V,J,e,n);
6: c2 <- (n/(n+2))*Lower(U,V,J,Lower(U,V,J,e,n+1),n);
7: X <- first two rows of MatrixFromColumns(c0,c1,c2);
8: {P0,P1,P2} <- Cancel3(X); Assert Simplify(X*{P0,P1,P2}^T)=0;
9: rho <-  $u^*(1-3*u-u^2)$ ;
10: C <-  $-9*n^4-36*n^3+6*n^2+84*n-13*n^3*u^4-12*u^3-6*u^2+3*u$ ;
11: Assert Simplify(P0*rho^2+n^2*P1*rho/(n+1)+n^2*P2/(n+2)-
  (C'*rho-(n+1)*C*rho'))=0;
12: a[0] <- 1; a[1] <- ExactQuotient(1,4-3*a[0]^2);
13: a[2] <- ExactQuotient(3*a[0]*a[1]^2,4-3*a[0]^2);
14: For k <- 1 to N-2:
15: a[k+2] <- ExactQuotient(-P0(k)*a[k]-P1(k)*a[k+1],P2(k));
16: S <- Sum(a[k]*x^k,k=0..N); Assert Coeffs(4*S-3-x-S^3,0..N)=0;
17: ODE <- MakeODE(P0,P1,P2);
18: Assert Coeffs(Substitute(ODE,A=S,A'=S',A''=S''),0..N-2)=0;
19: Return {a[0..N],Factor({P0,P1,P2}),R=C/rho,ODE=0};
```

6 Pseudocode operations and exact realization

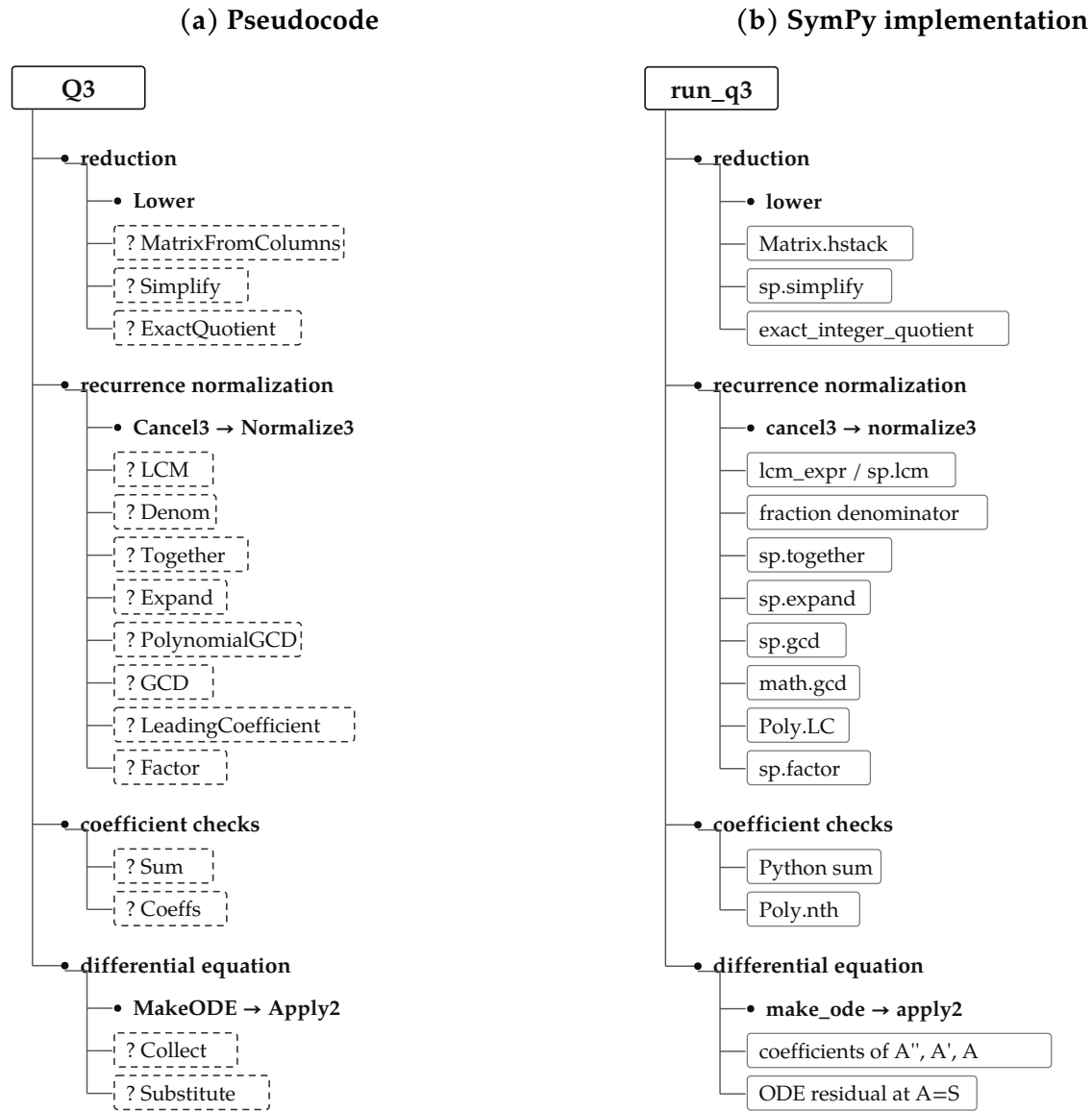


Figure 2. The local routines and their algebraic dependencies. Dashed leaves are operations used but not defined in the pseudocode; the corresponding rows on the right identify their exact realization.

7 Selected references

- [1] **Mary L. Boas**, *Mathematical Methods in the Physical Sciences*, third edition, Wiley, 2006. Useful here as an accessible reference for contour integrals, Cauchy’s integral formula, Laurent series, and residues: the complex-variable tools behind the coefficient and generating-function integrals.
- [2] **Philippe Flajolet and Robert Sedgewick**, *Analytic Combinatorics*, Cambridge University Press, 2009. DOI: [10.1017/CB09780511801655](https://doi.org/10.1017/CB09780511801655). Useful here for recursive tree specifications, ordinary generating functions, coefficient extraction, implicit algebraic equations, and the passage between combinatorial structures and analytic generating functions.
- [3] **George N. Raney**, “Functional composition patterns and power series reversion,” *Transactions of the American Mathematical Society* **94** (1960), 441–451. DOI: [10.1090/S0002-9947-1960-0114765-9](https://doi.org/10.1090/S0002-9947-1960-0114765-9). Useful for the rooted-tree and prefix-word form of the cycle argument, and for its connection with functional composition and power-series reversion.
- [4] **Nachum Dershowitz and Shmuel Zaks**, “The cycle lemma and some applications,” *European Journal of Combinatorics* **11** (1990), 35–40. DOI: [10.1016/S0195-6698\(13\)80053-4](https://doi.org/10.1016/S0195-6698(13)80053-4). Useful as a short, accessible presentation of the cycle lemma with explicit proofs and ordered-tree applications, making the historical result easier to verify independently.
- [5] **Gert Almkvist and Doron Zeilberger**, “The method of differentiating under the integral sign,” *Journal of Symbolic Computation* **10** (1990), 571–591. DOI: [10.1016/S0747-7171\(08\)80159-9](https://doi.org/10.1016/S0747-7171(08)80159-9). Useful for the continuous form of creative telescoping: derive a differential equation for a parameterized integral by differentiating the integrand and eliminating an exact derivative.
- [6] **Doron Zeilberger**, “The method of creative telescoping,” *Journal of Symbolic Computation* **11** (1991), 195–204. DOI: [10.1016/S0747-7171\(08\)80044-2](https://doi.org/10.1016/S0747-7171(08)80044-2). Useful for the general telescoper/certificate principle: a recurrence or differential operator is certified by an identity whose remaining term telescopes or integrates to zero.
- [7] **Alin Bostan, Shaoshi Chen, Frédéric Chyzak, and Ziming Li**, “Complexity of creative telescoping for bivariate rational functions,” in *Proceedings of ISSAC 2010*, ACM, 2010, 203–210. DOI: [10.1145/1837934.1837975](https://doi.org/10.1145/1837934.1837975). Useful as the closest algorithmic context for the rational-integrand reductions used here: it combines creative telescoping with Hermite reduction and treats the resulting telescopers and certificates computationally.
- [8] **Aaron Meurer et al.**, “SymPy: symbolic computing in Python,” *PeerJ Computer Science* **3** (2017), e103. DOI: [10.7717/peerj-cs.103](https://doi.org/10.7717/peerj-cs.103). Useful here as the reference for the exact computer-algebra system used in Appendix A: symbolic matrices, polynomial and rational simplification, exact differentiation, factorization, and coefficient checks are all realized in SymPy.
- [9] **OpenAI**, *ChatGPT*, official product documentation, <https://openai.com/academy/getting-started/>, accessed July 31, 2026. Useful as a provenance reference for the OpenAI conversational system associated with the pseudonymous coauthor credit “Mech.An.ika Sol (Open AI)” and the collaborative development of the human-readable certificate; it is not used as authority for any mathematical claim.
- [10] **Moonshot AI**, *Kimi*, official product page, <https://www.moonshot.cn/en/>, accessed July 31, 2026. Useful as a provenance reference for Kimi, which produced one of the independent one-shot implementations merged into the exact SymPy appendix.
- [11] **Anthropic**, *Claude*, official product page, <https://www.anthropic.com/claude>, accessed July 31, 2026. Useful as a provenance reference for Claude, which produced the other independent one-shot implementation merged into the exact SymPy appendix.

A Exact SymPy implementation

This implementation merges one-shot code produced independently by Kimi (Moonshot AI) and Claude (Anthropic AI), both working from the pseudocode and implementation presented in Sections 5–6.

A.1 Arithmetic and one-step lowering

The functions below define exact least common multiples and the map $w \mapsto (U - JV/m)w$.

```

1 #!/usr/bin/env python3
2 """Derive the A120590 recurrence, terms, and ODE from fixed q=3 data."""
3
4 from __future__ import annotations
5
6 import argparse
7 import math
8 from typing import Iterable
9
10 import sympy as sp
11
12
13 def lcm_expr(values: Iterable[sp.Expr]) -> sp.Expr:
14     result = sp.Integer(1)
15     for value in values:
16         result = sp.lcm(result, sp.simplify(value))
17     return result
18
19
20 def lower(U: sp.Matrix, V: sp.Matrix, J: sp.Matrix,
21          w: sp.Matrix, m: sp.Expr) -> sp.Matrix:
22     b = V * w
23     return sp.simplify(U * w - (J * b) / m)

```

A.2 Primitive recurrence normalization

This function clears rational denominators, removes common polynomial and integer factors, and fixes the sign of the recurrence vector.

```

26 def normalize3(p0: sp.Expr, p1: sp.Expr, p2: sp.Expr,
27               n: sp.Symbol) -> tuple[sp.Expr, sp.Expr, sp.Expr]:
28     values = [sp.together(p0), sp.together(p1), sp.together(p2)]
29     denoms = [sp.fraction(value)[1] for value in values]
30     common = lcm_expr(denoms)
31     q = [sp.expand(sp.cancel(common * value)) for value in values]
32
33     poly_gcd = sp.gcd(sp.gcd(q[0], q[1]), q[2])
34     if poly_gcd != 0:
35         q = [sp.expand(sp.cancel(value / poly_gcd)) for value in q]
36
37     coeff_denoms: List[sp.Expr] = []
38     for value in q:
39         coeff_denoms.extend(
40             sp.fraction(c)[1] for c in sp.Poly(value, n).all_coeffs()
41         )
42     common_coeff = lcm_expr(coeff_denoms)
43     q = [sp.expand(common_coeff * value) for value in q]
44
45     integer_coeffs: List[int] = []
46     for value in q:
47         integer_coeffs.extend(
48             abs(int(c)) for c in sp.Poly(value, n).all_coeffs() if c != 0
49         )
50     content = 0
51     for coeff in integer_coeffs:
52         content = math.gcd(content, coeff)
53     if content > 1:
54         q = [sp.expand(value / content) for value in q]
55
56     if sp.Poly(q[2], n).LC() < 0:
57         q = [-value for value in q]
58     return tuple(sp.factor(value) for value in q)

```

A.3 Kernel vector and recurrence-to-ODE map

The 2×3 minors form the kernel vector. The remaining functions convert quadratic recurrence coefficients into the coefficients of A'' , A' , and A .

```

61 def cancel3(X: sp.Matrix, n: sp.Symbol
62             ) -> tuple[sp.Expr, sp.Expr, sp.Expr]:
63     p0 = X[0, 1] * X[1, 2] - X[0, 2] * X[1, 1]
64     p1 = X[0, 2] * X[1, 0] - X[0, 0] * X[1, 2]
65     p2 = X[0, 0] * X[1, 1] - X[0, 1] * X[1, 0]
66     return normalize3(p0, p1, p2, n)
67
68
69 def apply2(P: sp.Expr, r: int, n: sp.Symbol, x: sp.Symbol,
70           A: sp.Symbol, Ap: sp.Symbol, App: sp.Symbol) -> sp.Expr:
71     poly = sp.Poly(sp.expand(P), n)
72     a = poly.coeff_monomial(n**2)
73     b = poly.coeff_monomial(n)
74     c = poly.coeff_monomial(1)
75     return (
76         a * (x**2 * App + (1 - 2 * r) * x * Ap + r**2 * A)
77         + b * (x * Ap - r * A)
78         + c * A
79     )
80
81
82 def make_ode(P0: sp.Expr, P1: sp.Expr, P2: sp.Expr,
83            n: sp.Symbol, x: sp.Symbol
84            ) -> tuple[sp.Expr, sp.Expr, sp.Expr]:
85     A, Ap, App = sp.symbols("A Ap App")
86     L = (
87         x**2 * apply2(P0, 0, n, x, A, Ap, App)
88         + x * apply2(P1, 1, n, x, A, Ap, App)
89         + apply2(P2, 2, n, x, A, Ap, App)
90     )
91     L = sp.expand(sp.cancel(L / x**2))
92     coeffs = (
93         sp.factor(L.coeff(App)),
94         sp.factor(L.coeff(Ap)),
95         sp.factor(L.coeff(A)),
96     )
97     if sp.Poly(coeffs[0], x).LC() < 0:
98         coeffs = tuple(-value for value in coeffs)
99     return coeffs

```

A.4 Certificate and term generation

The driver constructs X , derives the recurrence, verifies the exact certificate identity, and generates terms by exact division.

```

102 def exact_integer_quotient(number: sp.Expr, denom: sp.Expr) -> int:
103     value = sp.cancel(sp.simplify(number) / sp.simplify(denom))
104     if value.is_Integer is not True:
105         raise ArithmeticError(f"nonintegral quotient: ({number})/({denom})")
106     return int(value)
107
108
109 def run_q3(N: int) -> dict[str, object]:
110     if N < 2:
111         raise ValueError("N must be at least 2")
112
113     n, x = sp.symbols("n x")
114     U = sp.Rational(1, 13) * sp.Matrix(
115         [[153, 24, 3], [72, 9, 6], [0, 0, 0]]
116     )
117     V = sp.Rational(1, 13) * sp.Matrix(
118         [[-13, 0, 0], [75, 11, 3], [24, 3, 2]]
119     )
120     J = sp.Matrix([[0, 1, 0], [0, 0, 2], [0, 0, 0]])
121     e = sp.Matrix([1, 0, 0])
122
123     c0 = e
124     c1 = sp.simplify((n / (n + 1)) * lower(U, V, J, e, n))
125     c2 = sp.simplify(
126         (n / (n + 2)) * lower(U, V, J, lower(U, V, J, e, n + 1), n)
127     )
128     X = sp.simplify(sp.Matrix.hstack(c0, c1, c2)[:2, :])
129     P0, P1, P2 = cancel3(X, n)
130
131     if sp.simplify(X * sp.Matrix([P0, P1, P2])) != sp.zeros(2, 1):
132         raise AssertionError("X*P is not zero")
133
134     u = sp.symbols("u")
135     rho = u * (1 - 3 * u - u**2)
136     certificate_numerator = (
137         -9 * n * u**4 - 36 * n * u**3 + 6 * n * u**2
138         + 84 * n * u - 13 * n - 3 * u**4 - 12 * u**3
139         - 6 * u**2 + 3 * u
140     )
141     certificate_residual = sp.expand(
142         P0 * rho**2 + n * P1 * rho / (n + 1) + n * P2 / (n + 2)
143         - (sp.diff(certificate_numerator, u) * rho
144            - (n + 1) * certificate_numerator * sp.diff(rho, u))
145     )
146     if sp.factor(certificate_residual) != 0:
147         raise AssertionError("exact certificate identity failed")
148     certificate = sp.factor(certificate_numerator / rho)
149
150     terms = [0] * (N + 1)
151     terms[0] = 1
152     terms[1] = exact_integer_quotient(1, 4 - 3 * terms[0] ** 2)
153     terms[2] = exact_integer_quotient(
154         3 * terms[0] * terms[1] ** 2,
155         4 - 3 * terms[0] ** 2,
156     )
157     for k in range(1, N - 1):
158         numer = -P0.subs(n, k) * terms[k] - P1.subs(n, k) * terms[k + 1]
159         denom = P2.subs(n, k)
160         terms[k + 2] = exact_integer_quotient(numer, denom)

```

A.5 Series checks and command-line output

The remaining code checks the algebraic and differential equations and prints the certificate, recurrence, ODE, and requested terms.

```

162 S = sum(sp.Integer(terms[k]) * x**k for k in range(N + 1))
163 cubic = sp.Poly(sp.expand(4 * S - 3 - x - S**3), x)
164 if any(cubic.nth(k) != 0 for k in range(N + 1)):
165     raise AssertionError("truncated series fails the cubic check")
166
167 ode = make_ode(P0, P1, P2, n, x)
168 residual = sp.Poly(
169     sp.expand(ode[0] * sp.diff(S, x, 2)
170         + ode[1] * sp.diff(S, x) + ode[2] * S),
171     x,
172 )
173 if any(residual.nth(k) != 0 for k in range(N - 1)):
174     raise AssertionError("truncated series fails the ODE check")
175
176 return {
177     "n": n,
178     "x": x,
179     "X": X,
180     "P": (P0, P1, P2),
181     "certificate": certificate,
182     "terms": terms,
183     "ode": ode,
184 }
185
186
187 def main() -> None:
188     parser = argparse.ArgumentParser()
189     parser.add_argument("N", nargs="?", type=int, default=30)
190     args = parser.parse_args()
191
192     result = run_q3(args.N)
193     n = result["n"]
194     P0, P1, P2 = result["P"]
195     c2, c1, c0 = result["ode"]
196
197     print("A120590 ternatree q=3")
198     print(f"N = {args.N}")
199     print("X*P check: PASS")
200     print("exact certificate identity: PASS")
201     print("exact recurrence divisions: PASS")
202     print("cubic series check: PASS")
203     print("ODE series check: PASS")
204     print()
205     print("Factored recurrence coefficients:")
206     print(f"P0(n) = {P0}")
207     print(f"P1(n) = {P1}")
208     print(f"P2(n) = {P2}")
209     print()
210     print("Certificate:")
211     print(f"R(n,u) = {result['certificate']}")
212     print()
213     print("Expanded recurrence coefficients:")
214     print(f"P0(n) = {sp.expand(P0)}")
215     print(f"P1(n) = {sp.expand(P1)}")
216     print(f"P2(n) = {sp.expand(P2)}")
217     print()
218     print("ODE:")
219     print(f"({c2})*A''(x) + ({c1})*A'(x) + ({c0})*A(x) = 0")
220     print(f"A(0) = 1, A'(0) = 1")
221     print()
222     print("Terms:")
223     for index, value in enumerate(result["terms"]):
224         print(f"a({index}) = {value}")
225
226
227 if __name__ == "__main__":
228     main()

```
